ARIMA uses a distributed lag model in which regressions are used to predict future values based on lagged values (an *autoregressive* process). Autoregressive and moving average parameters are used to tune the model, along with *polynomial factor differencing*—a process used to make the series *stationary* (i.e., having constant statistical properties over time, such as mean and variance), a condition that ARIMA requires the input series to have.

In this example, we attempt to perform anomaly detection on per-minute metrics of a host's CPU utilization.[14] The *y*-axis of Figure 3-2 shows the percentage CPU utilization, and the *x*-axis shows time.

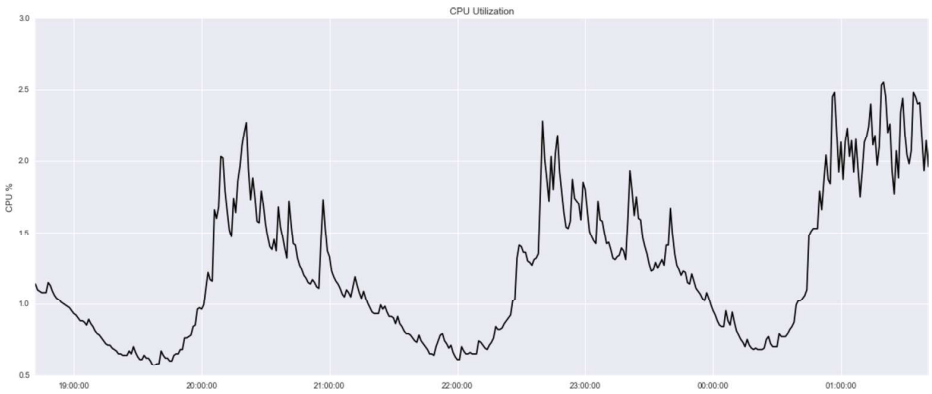

Figure 3-2. CPU utilization over time

We can observe a clear periodic pattern in this series, with peaks in CPU utilization roughly every 2.5 hours. Using a convenient time series library for Python, PyFlux, we apply the ARIMA forecasting algorithm with autoregressive (AR) order 11, moving average (MA) order 11, and a differencing order of 0 (because the series looks stationary).[15] There are some tricks to determining the AR and MA orders and the differencing order, which we will not elaborate on here. To oversimplify matters, AR and MA orders are needed to correct any residual autocorrelations that remain in the differenced series (i.e., between the time-shifted series and itself). The differencing order is a term used to make the series stationary—an already stationary series should have a differencing order of 0, a series with a constant average trend (steadily trending upward or downward) should have a differencing order of 1, and a series with a time-varying trend (a trend that changes in velocity and direction over the

series) should have a differencing order of 2. Let's plot the in-sample fit to get an idea of how the algorithm does:[16]

```
import pandas as pd
import pyflux as pf
from datetime import datetime

# Read in the training and testing dataset files
data_train_a = pd.read_csv('cpu-train-a.csv',
    parse_dates=[0], infer_datetime_format=True)
data_test_a = pd.read_csv('cpu-test-a.csv',
    parse_dates=[0], infer_datetime_format=True)

# Define the model
model_a = pf.ARIMA(data=data_train_a,
                   ar=11, ma=11, integ=0, target='cpu')

# Estimate latent variables for the model using the
# Metropolis-Hastings algorithm as the inference method
x = model_a.fit("M-H")

# Plot the fit of the ARIMA model against the data
model_a.plot_fit()
```

Figure 3-3 presents the result of the plot.

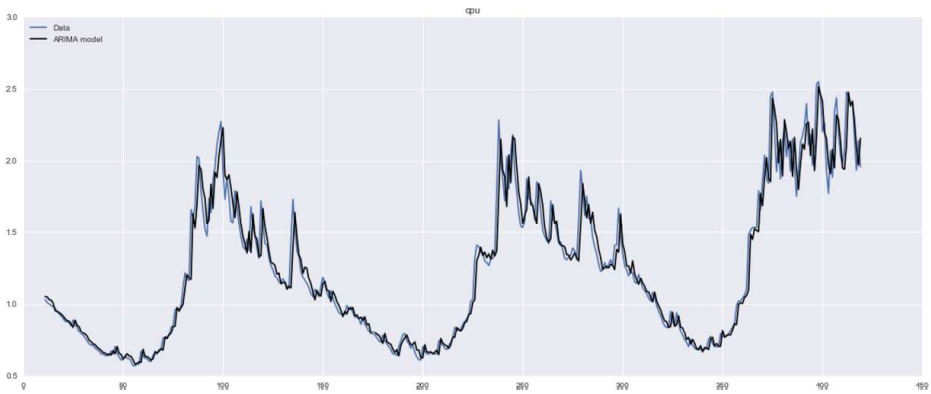

Figure 3-3. CPU utilization over time fitted with ARIMA model prediction

As we can observe in Figure 3-3, the results fit the observed data quite well. Next, we can do an in-sample test on the last 60 data points of the training data. The in-sample test is a validation step that treats the last subsection of the series as unknown and

performs forecasting for those time steps. This process allows us to evaluate performance of the model without running tests on future/test data:

```
> model_a.plot_predict_is(h=60)
```

The in-sample prediction test (depicted in Figure 3-4) looks pretty good because it does not deviate from the original series significantly in phase and amplitude.

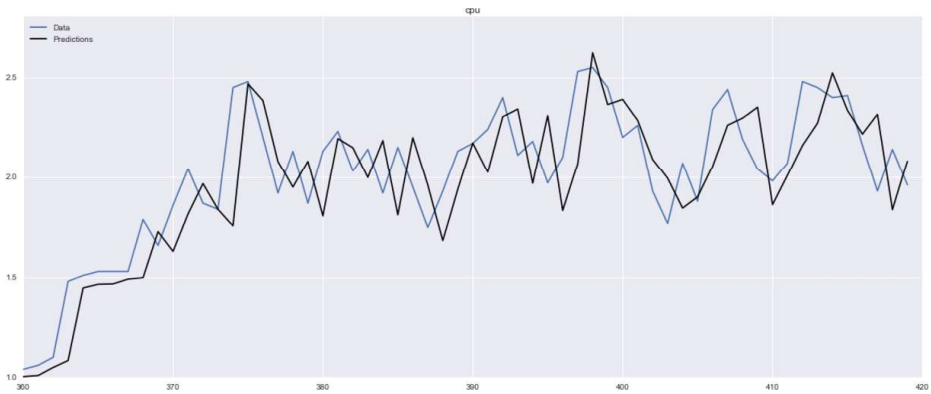

Figure 3-4. In-sample (training set) ARIMA prediction

Now, let's run the actual forecasting, plotting the most recent 100 observed data points followed by the model's 60 predicted values along with their confidence intervals:

```
> model_a.plot_predict(h=60, past_values=100)
```

Bands with a darker shade imply a higher confidence; see Figure 3-5.

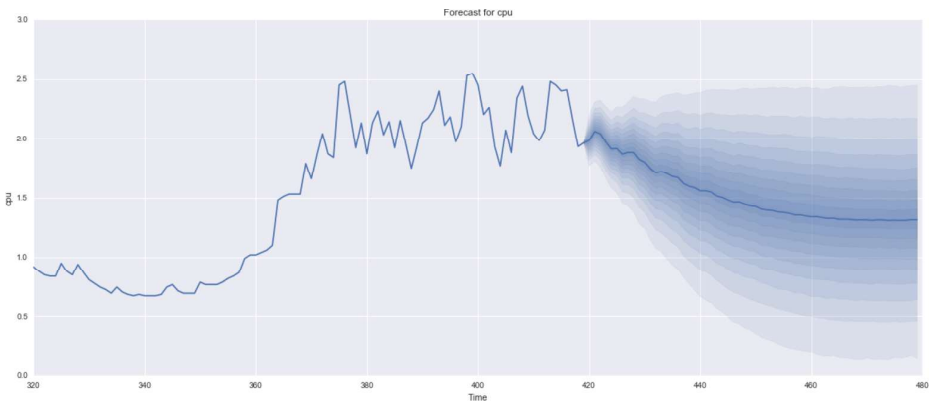

Figure 3-5. Out-of-sample (test-set) ARIMA prediction

Comparing the prediction illustrated in Figure 3-5 to the actual observed points illustrated in Figure 3-6, we see that the prediction is spot-on.

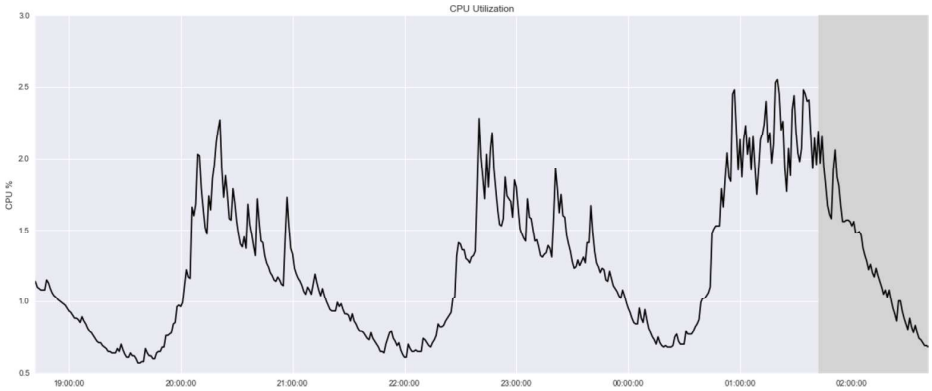

Figure 3-6. Actual observed data points

To perform anomaly detection using forecasting, we compare the observed data points with a rolling prediction made periodically. For example, an arbitrary but sensible system might make a new 60-minute forecast every 30 minutes, training a new ARIMA model using the previous 24 hours of data. Comparisons between the forecast and observations can be made much more frequently (e.g., every three minutes). We can apply this method of *incremental learning* to almost all the algorithms that we will discuss, which allows us to approximate streaming behavior from algorithms originally designed for batch processing.

Let's perform the same forecasting operations on another segment of the CPU utilization dataset captured at a different time:

```
data_train_b = pd.read_csv('cpu-train-b.csv',
    parse_dates=[0], infer_datetime_format=True)
data_test_b = pd.read_csv('cpu-test-b.csv',
    parse_dates=[0], infer_datetime_format=True)
```

Forecasting using the same *ARIMAX* model[17] trained on `data_train_b`, the prediction is illustrated in Figure 3-7.

17 ARIMAX is a slight modification of ARIMA that adds components originating from standard econometrics, known as explanatory variables, to the prediction models.

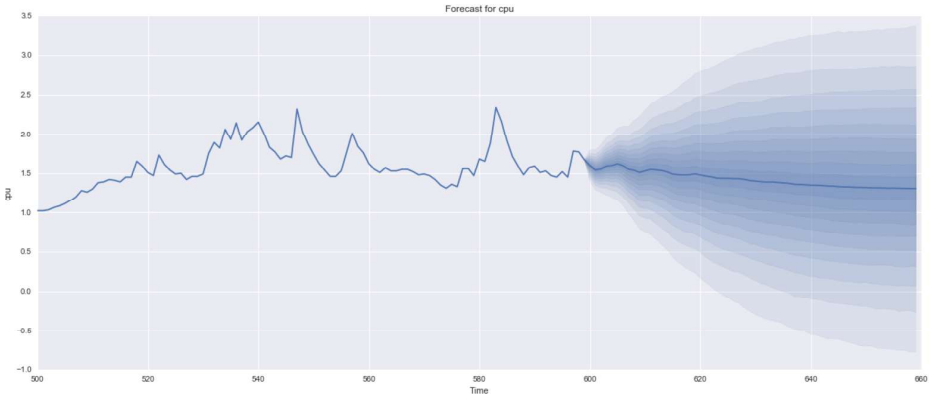

Figure 3-7. Out-of-sample (test-set, data_train_b) ARIMAX prediction

The observed values are, however, very different from the predictions illustrated in Figure 3-8.

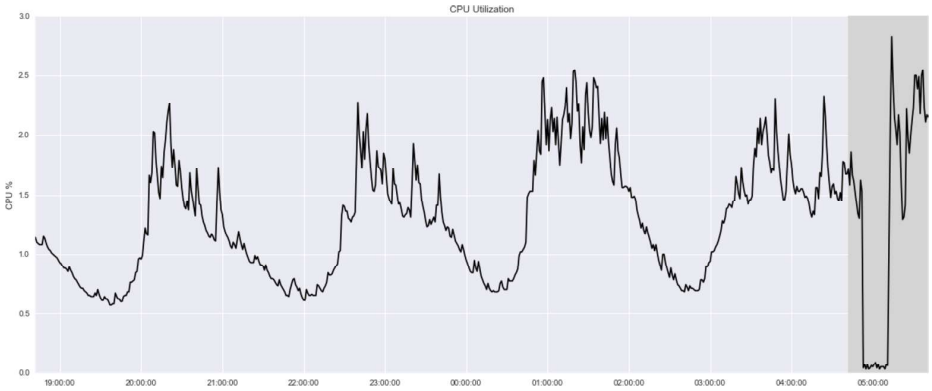

Figure 3-8. Actual observed data points (data_train_b)

We see a visible anomaly that occurs a short time after our training period. Because the observed values fall within the low-confidence bands, we will raise an anomaly alert. The specific threshold conditions for how different the forecasted and observed series must be to raise an anomaly alert is something that is highly application specific but should be simple enough to implement on your own.